

SEGG-U1-P1 — OWASP TOP 10: INVESTIGACIÓN Y ANÁLISIS DE VULNERABILIDADES

Seguridad en el Desarrollo de Aplicaciones

Ricardo Reséndiz González

IDGS17

INTRODUCCIÓN

El objetivo de este reporte es investigar y analizar las 10 categorías principales de riesgos de seguridad en aplicaciones web, tomando como base la documentación oficial de OWASP Top 10. La práctica solicita una tabla comparativa, casos reales, ejemplos de código vulnerable y seguro, un ranking personal de criticidad y una conclusión final.

Nota honesta: la guía menciona OWASP Top 10 2023, pero OWASP no maneja una versión oficial independiente con ese año para esta lista. Por compatibilidad con la estructura A01-A10 solicitada en la práctica, se utiliza la lista OWASP Top 10:2021, que es la referencia académica y técnica más usada para estas categorías.

TABLA COMPARATIVA DE VULNERABILIDADES OWASP TOP 10

Clave	Vulnerabilidad	Descripción corta	Ejemplo de ataque	Impacto potencial	Mitigación
A01	Broken Access Control	Ocurre cuando la aplicación no valida correctamente qué puede ver o hacer cada usuario.	Un usuario cambia el id en la URL y consulta información de otra cuenta.	Robo de datos, modificación no autorizada, escalamiento de privilegios o toma de cuentas.	Aplicar autorización en servidor, negar por defecto, validar propiedad del recurso y usar pruebas de control de acceso.
A02	Cryptographic Failures	Fallas al proteger datos sensibles con cifrado, hash seguro, TLS o manejo correcto de llaves.	Contraseñas guardadas con SHA-1 sin salt o datos sensibles transmitidos sin TLS.	Exposición de contraseñas, tokens, datos personales o información financiera.	Usar TLS, algoritmos actuales, bcrypt/Argon2 para contraseñas, rotación de llaves y no almacenar secretos en código.
A03	Injection	Datos no confiables llegan a consultas, comandos o intérpretes sin validación suficiente.	Concatenar el correo del usuario dentro de una consulta SQL de login.	Lectura o modificación de bases de datos, ejecución remota de código o fuga masiva de información.	Usar consultas parametrizadas, ORM seguro, validación de entrada y escape contextual.
A04	Insecure Design	El problema nace en el diseño: faltan controles de negocio, límites o validaciones de seguridad desde la arquitectura.	Un endpoint permite operaciones financieras sin validar reglas del negocio completas.	Fraude, abuso de funcionalidades, omisión de controles y fallas que no se arreglan solo parchando código.	Modelado de amenazas, requisitos de seguridad, historias de abuso, validación de reglas y revisión de arquitectura.
A05	Security Misconfiguration	Configuraciones inseguras en servidores, nube, frameworks, APIs, permisos o encabezados de seguridad.	Un bucket o API queda público por configuración incorrecta.	Exposición de datos, acceso a paneles internos o ampliación del ataque.	Hardening, configuración automatizada, mínimos privilegios, revisión periódica y ambientes separados.
A06	Vulnerable and Outdated Components	Uso de librerías, frameworks, servidores o dependencias con vulnerabilidades conocidas o sin mantenimiento.	Una aplicación conserva una versión vulnerable de Apache Struts o Log4j.	RCE, robo de datos y compromiso total de la aplicación por fallas ya conocidas.	Inventario de dependencias, SCA, actualización continua, SBOM y parches críticos rápidos.
A07	Identification and Authentication Failures	Errores en login, sesiones, recuperación de contraseña, MFA o protección contra credenciales robadas.	Ataques de credential stuffing contra cuentas sin MFA obligatorio.	Toma de cuentas, fraude, acceso a información privada y movimiento lateral.	MFA, rate limiting, bloqueo inteligente, contraseñas seguras, sesiones protegidas y monitoreo de inicios anómalos.
A08	Software and Data Integrity Failures	La aplicación confía en actualizaciones, scripts, paquetes o datos sin verificar integridad.	Un script de CI/CD usado por clientes es modificado para robar variables de entorno.	Ataques de cadena de suministro, inserción de código malicioso y compromiso de muchos clientes a la vez.	Firmas digitales, repositorios confiables, revisión de cambios, protección CI/CD y verificación de artefactos.
A09	Security Logging and Monitoring Failures	La aplicación no registra, alerta o responde bien ante eventos importantes de seguridad.	Alertas de malware o accesos extraños se generan, pero nadie las atiende.	El atacante permanece semanas o meses sin ser detectado; se pierde evidencia forense.	Logs útiles, SIEM, alertas accionables, monitoreo de eventos críticos y plan de respuesta a incidentes.

Clave	Vulnerabilidad	Descripción corta	Ejemplo de ataque	Impacto potencial	Mitigación
A10	Server-Side Request Forgery (SSRF)	La aplicación solicita recursos externos usando URLs controladas por el usuario sin validación adecuada.	Un atacante obliga al servidor a consultar metadatos internos de la nube.	Robo de credenciales internas, acceso a servicios privados y movimiento dentro de la red.	Allow-list de destinos, bloqueo de IPs internas, validación estricta de URLs y segmentación de red.

CASOS REALES POR VULNERABILIDAD

Clave	Empresa / producto	Año	Cómo fue explotada
A01	Facebook / Meta	2018	La función View As permitió exponer tokens de acceso. Un actor externo aprovechó la interacción de varios bugs para obtener tokens y acceder a cuentas.
A01	Twitter / X	2022	Un bug de API permitía consultar si un correo o teléfono estaba asociado a una cuenta, aun cuando esa relación debía mantenerse privada.

CÓDIGO VULNERABLE VS CÓDIGO SEGURO

A03 Injection - Login con SQL

```
1 // Vulnerable: cualquier usuario autenticado puede consultar cualquier id
2 app.get('/api/usuarios/:id', auth, async (req, res) => {
3   const usuario = await Usuario.findById(req.params.id);
4   res.json(usuario);
5 });
6
7 // Seguro: valida propiedad del recurso o rol administrador
8 app.get('/api/usuarios/:id', auth, async (req, res) => {
9   const esPropietario = req.user.id === req.params.id;
10  const esAdmin = req.user.rol === 'admin';
11
12  if (!esPropietario && !esAdmin) {
13    return res.status(403).json({ error: 'Acceso denegado' });
14  }
15
16  const usuario = await Usuario.findById(req.params.id);
17  res.json(usuario);
18 });
```

A01 Broken Access Control - Consulta de perfil

```
1 // Vulnerable: usa hash débil y sin salt
2 const hash = sha1(password);
3 await Usuario.create({ correo, password_hash: hash });
4
5 // Seguro: usa algoritmo para contraseñas con salt y costo
6 const hash = await bcrypt.hash(password, 12);
7 await Usuario.create({ correo, password_hash: hash });
8
```

A02 Cryptographic Failures – Contraseñas

```
1 // Vulnerable: concatena datos del usuario directamente en SQL
2 const sql = "SELECT * FROM usuarios WHERE correo = '" + correo + "' AND password = '" + password + "'";
3 const usuario = db.query(sql);
4
5 // Seguro: usa consulta parametrizada y contraseña con hash
6 const sql = "SELECT * FROM usuarios WHERE correo = ?";
7 const usuario = db.query(sql, [correo]);
8 const valido = await bcrypt.compare(password, usuario.password_hash);
```

RANKING PERSONAL DE LAS 5 VULNERABILIDADES MÁS CRÍTICAS

1. A01 Broken Access Control

- La pongo en primer lugar porque es muy común y muchas veces no se ve a simple vista. Si un sistema permite que un usuario vea o modifique recursos ajenos, el daño puede ser directo: fuga de datos, fraude o cambios no autorizados.

2. A03 Injection

- Sigue siendo crítica porque puede convertir una entrada de texto en acceso a base de datos o ejecución de comandos. Aunque ya existen defensas conocidas, sigue apareciendo cuando se programa con prisas o sin consultas parametrizadas.

3. A06 Vulnerable and Outdated Components

- Hoy casi todos los proyectos dependen de librerías externas. Si no se actualizan, una falla pública puede afectar a miles de sistemas, como pasó con Struts y Log4j.

4. A07 Identification and Authentication Failures

- Una mala autenticación abre la puerta a toma de cuentas. En sistemas reales, el riesgo aumenta porque muchos usuarios reutilizan contraseñas y no siempre activan MFA.

5. A08 Software and Data Integrity Failures

- Una mala autenticación abre la puerta a toma de cuentas. En sistemas reales, el riesgo aumenta porque muchos usuarios reutilizan contraseñas y no siempre activan MFA.

CONCLUSIÓN

Conocer el OWASP Top 10 es importante porque aterriza la seguridad a problemas que sí pasan en sistemas reales. Como desarrollador, muchas veces uno se enfoca en que la aplicación funcione, que el login entre, que la consulta regrese datos o que el botón haga lo que debe hacer. El problema es que una aplicación puede funcionar bien para el usuario normal y aun así estar mal protegida. Por ejemplo, si no valido permisos en el backend, si concateno datos en una consulta SQL, si dejo una dependencia vieja o si guardo contraseñas de forma débil, el sistema puede quedar expuesto aunque visualmente parezca correcto.

Esta práctica ayuda a ver que la seguridad no es algo que se agrega al final, sino algo que se debe pensar desde el diseño, la programación, las pruebas y el despliegue. También deja claro que muchos incidentes grandes no ocurrieron por ataques mágicos o imposibles de entender, sino por errores bastante concretos: parches no aplicados, configuraciones públicas, falta de monitoreo, validaciones incompletas o confianza excesiva en componentes externos. En mi práctica profesional, este conocimiento se puede aplicar revisando mejor los endpoints, validando que cada usuario solo acceda a lo que le corresponde, usando consultas parametrizadas, protegiendo secretos, actualizando dependencias y dejando logs que realmente ayuden a detectar problemas.

REFERENCIAS

OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/2021/>

National Institute of Standards and Technology. (s.f.). National Vulnerability Database. <https://nvd.nist.gov/>

Cybersecurity and Infrastructure Security Agency. (s.f.). Cybersecurity Advisories. <https://www.cisa.gov/news-events/cybersecurity-advisories>

Have I Been Pwned. (s.f.). Data breach database. <https://haveibeenpwned.com/>

UpGuard. (s.f.). Data breach research and security reports. <https://www.upguard.com/breaches>